

React + API Integration Technical Assessment

Build a React.js User Management Dashboard that consumes a public REST API.

API: JSONPlaceholder — <https://jsonplaceholder.typicode.com>

1. API Integration

- Fetch users from **/users**.
- Display Name, Email, Phone, Company and Website.
- Implement loading state while the API is being called.
- Handle API failures with a clear error state.
- Do not hardcode user data.

2. Search & Filtering

- Search users by name and email.
- Filter users by company.
- Show a proper “No users found” message.
- Search/filtering should work smoothly without unnecessarily calling the API on every keystroke.

3. User Details

- When a user is selected, open a modal or separate details section.
- Display complete user information.
- Fetch the user's posts using **/posts?userId={userId}** and display them.

4. Create User

- Create a form with Name, Email, Phone, Website and Company name.
- Submit using **POST /users**.
- Validate required fields and show validation errors.
- Show loading, API error and success states.
- Do not refresh the entire page after submission.

5. Edit User

- Provide an Edit option.
- Pre-fill the existing user data.
- Submit changes using **PUT /users/{id}**.
- Show loading, success and error states.

6. Delete User

- Provide a Delete option.
- Ask for confirmation before deleting.
- Call **DELETE /users/{id}**.
- After success, remove the user from the UI and show a success message.

7. React Requirements

- Use functional components and React Hooks.
- Use proper state management and `useEffect`.
- Create reusable components.
- Separate API/service logic from UI components.
- Avoid unnecessary re-renders.
- Handle asynchronous operations properly.
- Axios or Fetch may be used. Redux/Context is optional and should only be used when justified.

8. UI Requirements

- Clean, responsive and easy-to-understand UI.
- Consistent spacing and usable desktop/mobile layout.
- Tailwind CSS, CSS Modules or normal CSS may be used.

9. Suggested Code Structure

- `src/components/` — `UserList`, `UserCard`, `UserForm`, `UserDetails`, `ConfirmationModal`
- `src/services/` — `userApi.js`
- `src/pages/` — `Users.jsx`
- `src/hooks/` — custom hooks if needed
- `src/utils/` — utility functions
- `src/App.jsx`

10. Bonus Points

- Pagination
- Debounced search
- Sorting
- React Query
- Optimistic UI updates
- Reusable custom hooks
- Unit tests
- Accessibility
- Toast notifications

11. Submission Requirements

- Push the complete project to a GitHub repository.
- Include a clear `README.md` with setup and run instructions.
- Deploy the application on Vercel.
- Share both the GitHub repository link and the live Vercel deployment link.
- Make sure the deployed application is working and accessible.

- Mention any assumptions or known limitations in the README.

12. Time Limit

- **3–4 hours.**
- Priority should be: API Integration → React Architecture → Error Handling → State Management → Code Quality → UI.